

LLM OVERVIEW SLIDES | LLM & RAG GUIDE

LLM OVERVIEW SLIDES | LLM & RAG GUIDE

LLM 101 - THIS SLIDE SESSION WALK THRU LLM APPLICATIONS, ARCHITECTURE AND BUILDING BLOCKS. SLIDES DESCRIBE OPEN SOURCE VS CLOSED LLM AND CHOOSE. SLIDES ALSO FOCUS ON LLM STEERABILITY. SESSION SLIDES ALSO DESCRIBE PROMPT ENGINEERING, TOKENIZATION AND FACTORS TO CONSIDER BUILDING APPLICATIONS USING LLM. AT LAST IT DESCRIBE PROCESS ARCHITECTURE, COMPONENT ARCHITECTURE AND HOW TO ESTIMATE COST OF LLM.

AGENDA



Data
Knobs

AGENDA

LLM TOPICS

LLM Topics

Large Language Model Guide



Data
Knobs

[LLM TOPICS](#)

LLM OVERVIEW

Large Language Model

A large language model (LLM) is a type of artificial intelligence (AI) that is trained on a massive dataset of text and code. This allows the model to learn the statistical relationships between words and phrases, which in turn allows it to generate text, translate languages, write different kinds of creative content, and answer your questions in an informative way.

Here are common LLMs

- GPT3.5 and GPT4
- Bard
- LLama, Llama 2 - 65 B parameters
- BERT Large by Hugging Face - 340M parameters

LLM is new IP

Data
Knobs

[LLM OVERVIEW](#)

APPLICATIONS OF LLM

Applications of LLM

- Chatbots
- Translation
- Text Summarization
- Text Generation/Content Writing
- NLP related tasks
- Data Analysis
- Code generation
- Automation and workflow building
- CSS generation



[APPLICATIONS OF LLM](#)

NLP APPLICATIONS

Applications - LLMs Impact on NLP Tasks

State of art results on

- Text classification,
- Sentiment analysis
- Translations
- SEO

Ability to automate NLP tasks

- Text completion
- Content /Article generation
- Summarize document
- Q&A Bots



[NLP APPLICATIONS](#)

SOFTWARE APPLICATIONS

Applications - LLMs for Software Development

Ability to assist code writing

- NLP to SQL
- Code generation for Engineering
- Web design generation (see www.kreatewebsites.com)
- Identify bugs in code and produce solution



[SOFTWARE APPLICATIONS](#)

EVOLVING APPLICATIONS OF LLMS

Applications of LLM

Today	Evolving
✓ Write articles	✓ Create Ideas for new product
✓ Summarize Text.	✓ Reimagine user experience
✓ Generate art, images	✓ Redefine and re invent workflow
✓ Generate code examples	✓ Simulations for better decision making
✓ Build chatbots and AI Assistants	✓ Drug discovery
✓ Build new design	✓ Generate data for Reinforcement Learning, detect cyber attack.
✓ Build E2e Websites	✓ Interior Decoration and Film production



[EVOLVING APPLICATIONS OF LLMS](#)

OPEN SOURCE VS CLOSE

LLMs - Open Source vs Closed Model

- Open-source models are made available to the public for free. The underlying code and often the trained models can be accessed, used, and modified under the terms of their respective open-source licenses. Examples include earlier models like GPT-2 by OpenAI.
 - a. [LLama](#)
 - b. [Hugging Face](#)
- Closed models, on the other hand, are proprietary and usually not freely accessible to the public. Access to these models may be restricted, with use often tied to a paid service. An example of this is GPT-3, which, while not completely closed (the research paper and some model details are available), is only accessible through a paid API provided by OpenAI.
 - a. [GPT3.5, GPT 4](#)
 - b. [Bard](#)



[OPEN SOURCE VS CLOSE](#)

BUILDING BLOCKS OF LLMS

LLM Building Blocks

- **Data:** LLMs are trained on massive datasets of text and code. The quality and size of the dataset can have a significant impact on the performance of an LLM.
- **Architecture:** The architecture of an LLM refers to the way that it is designed. Different architectures have different strengths and weaknesses.
- **Training:** LLMs are trained using a process called supervised learning. This involves feeding the LLM a large dataset of text and code, along with the correct output for each input.
- **Inference:** Inference is the process of using an LLM to generate output. This involves providing the LLM with a prompt and then using the LLM's knowledge of language to generate the desired output.

Above topics are of interest to model builder. For Model consumer one of most important factor is

- **Model parameter:** It is the total number of variables that the model needs to learn in order to generate text, translate languages, and answer questions.
 - a. LLAMA - 65B parameters
 - b. GPT 3 - 175B parameters
 - c. GPT4 - 100 Trillion parameters



[BUILDING BLOCKS OF LLMS](#)

MULTI MODEL LLMS

Multi Modal LLM

Multimodal capability in LLMs refers to their ability to process and understand information from multiple modalities, such as text, images, audio, and video. This allows them to perform tasks that would be difficult or impossible for unimodal models, such as:

- Generating multimodal outputs: LLMs can generate outputs that combine multiple modalities, such as captions for images, videos with narration, or musical pieces with lyrics.
- Reasoning about multimodal information: LLMs can use information from multiple modalities to reason about the world. For example, they can use text and images to understand the layout of a room, or they can use text and audio to identify the speaker of a sentence.
- Learning from multimodal data: LLMs can learn from data that is presented in multiple modalities. For example, they can be trained on a dataset of text and images to learn how to identify objects in images.



[MULTI MODEL LLMS](#)

COMMON TERMINOLOGY LLMS

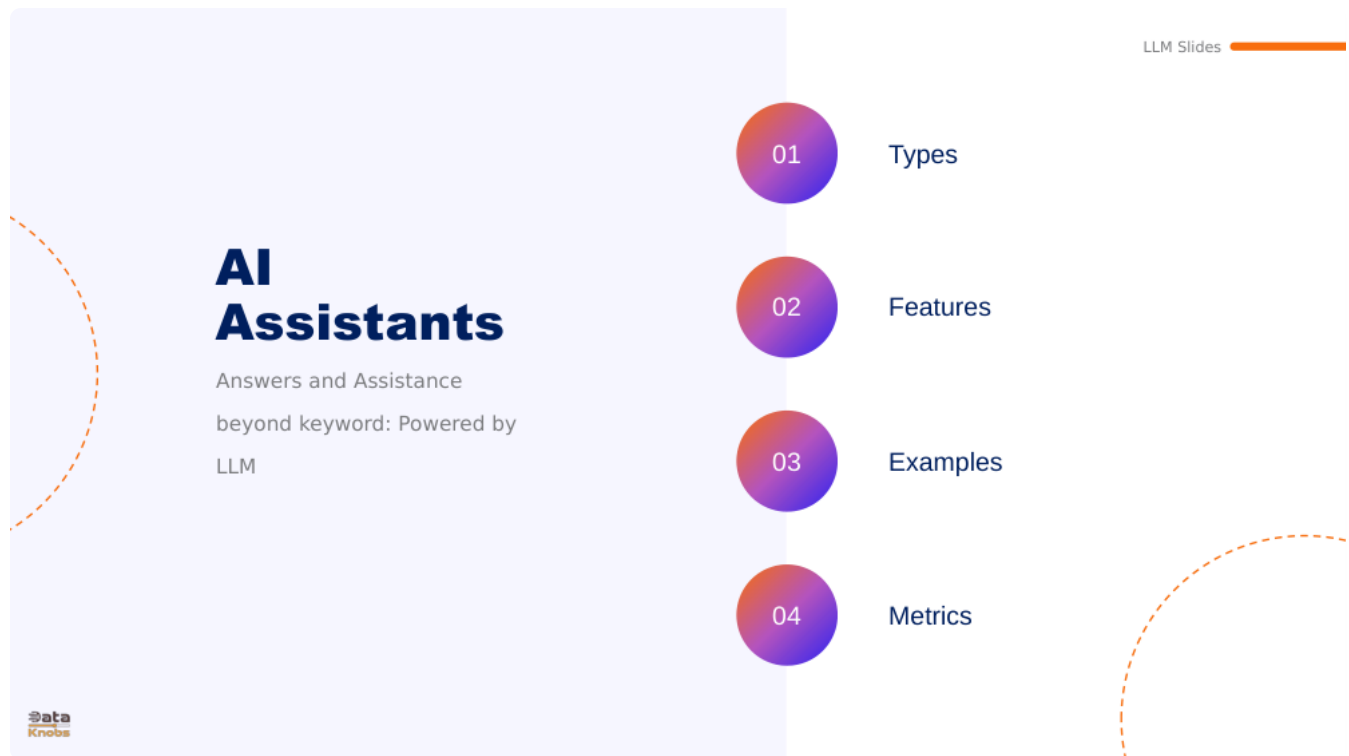
Common Terms

Embeddings	A set of models that can convert text into a numerical form
Tokenization	Process of breaking down text into smaller units called tokens. LLMs typically use subword tokenization, which splits words into subword components to handle different languages
Moderation	A fine-tuned model that can detect whether text may be sensitive or unsafe
Foundation Model	Foundation models are like the "learning blocks" of AI, trained on enormous amounts of unlabeled data to perform a wide range of tasks from translating languages to writing different kinds of creative content.
RAG	Retrieval Augmentation Generation
Perplexity	Metric used to measure how well an LLM can predict the next word in a sequence. A lower perplexity indicates that the LLM is better at understanding and generating natural language
Fine Tuning	This is the process of adapting a pre-trained LLM to a specific task by training it on a smaller dataset relevant to that task.



[COMMON TERMINOLOGY LLMS](#)

AI ASSISTANTS TYPES



[AI ASSISTANTS TYPES](#)

TYPES OF AI ASSISTANTS

Types of AI Assistant and Agent



TYPES OF AI ASSISTANTS

FEATURES OF AI ASSISTANTS

Features, KB and Algorithms - Types of AI Assistant

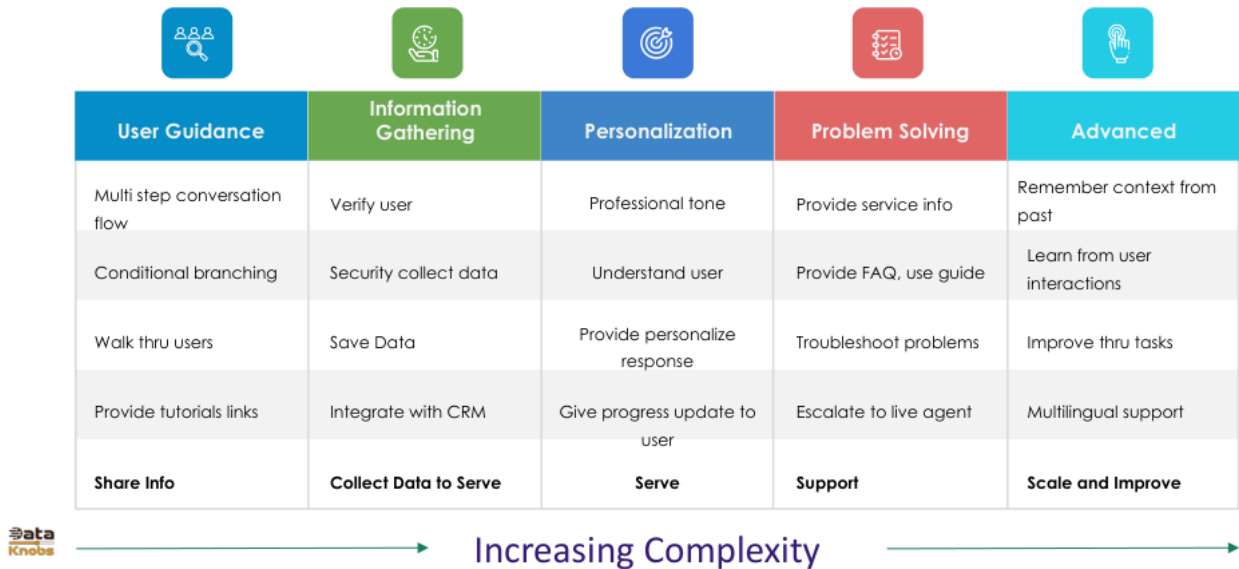
Search	Answer	Recommend	Plan	Execute
Find information	Provide facts	Suggest relevant items	Create Reports	Execute tasks
Use knowledge base	Use upto date KB	Use user Preference	Use user goals	Use integration
NLP & Search Algorithm	LLMs	User behavior modeling	Advance AI Reasoning	Automate
Travel Website	Tax or Legal Research	Which movies to watch	Financial Plan	Website Creation

Increasing Complexity

FEATURES OF AI ASSISTANTS

EXAMPLE FEATURES OF AI ASSISTA

Example Features - LLM Chatbot

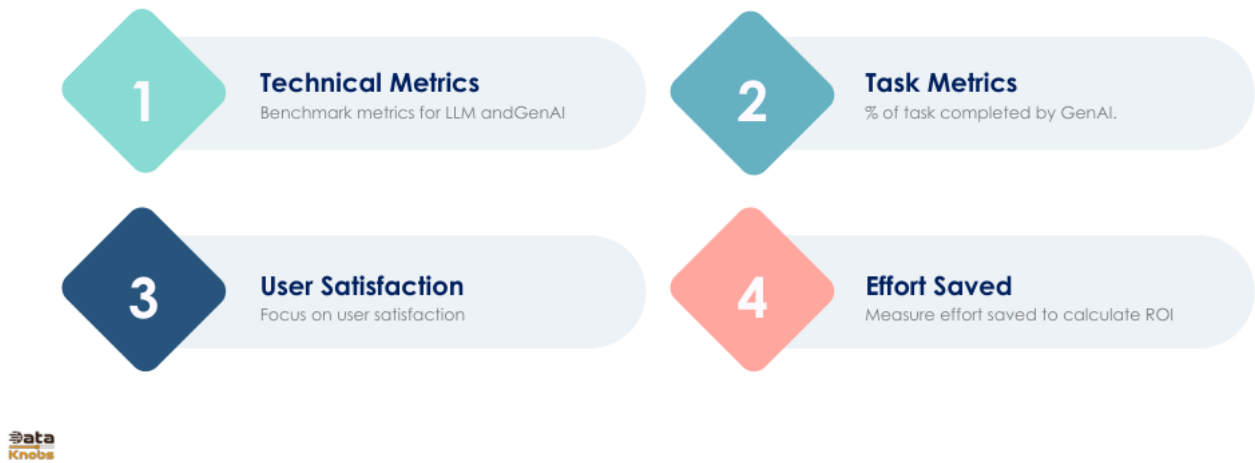


EXAMPLE FEATURES OF AI ASSISTA

AI ASSISTANT EVALUATION METRIC

LLM Evaluation Metrics

Evaluating a LLM Model is complex



AI ASSISTANT EVALUATION METRIC

ASSISTANT BOT METRICS

Metrics for AI Assistants

How effective AI chatbot responses are

01

Task and goal completion

Did user completed the task and met goal

02

Satisfaction Rate

Is user satisfied the response

03

Conversation Length

How many steps it took to reach closure

04

Self Service Rate

How many users were able to self serve

05

Engagement Rate

How often user interact and come back.

Data
Knobs

ASSISTANT BOT METRICS

METRICS TO EVALUATE AI ASSISTA

Metrics for AI Assistant Individual Response

Focus on each response effectiveness



Data
Knobs

METRICS TO EVALUATE AI ASSISTA

TECHNICAL METRICS AI ASSISTANT

BLEU Score

1

Measure of similarity between generated text and reference text

GLEU Score

2

GLEU Score avg the accuracy of model on each task

ROUGH

3

ROUGH score sentences based on n-gram overlap

METEOR

4

METEOR incorporate both n-gram matching and synonym to evaluate semantic similarity

Perplexity

5

Perplexity measure how well a language model predict next word

Technical Metrics to Evaluate LLM

Decoding the mystery of LLM



[TECHNICAL METRICS AI ASSISTANT](#)

METRICS FOR SEARCH BOT

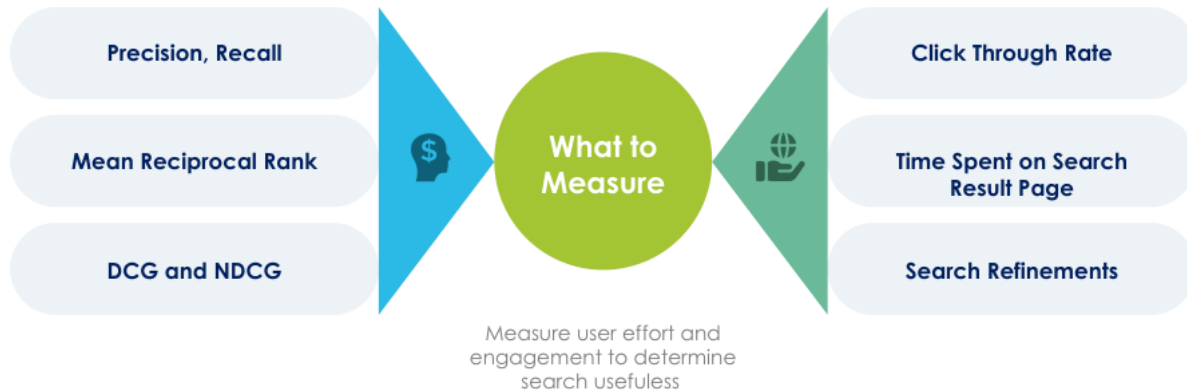
Metrics for Search - Get context

How well search logic retrieves information related to user query

Technical Metric

Behavioral Metric

Measure Search Effectiveness



Data
Knobs

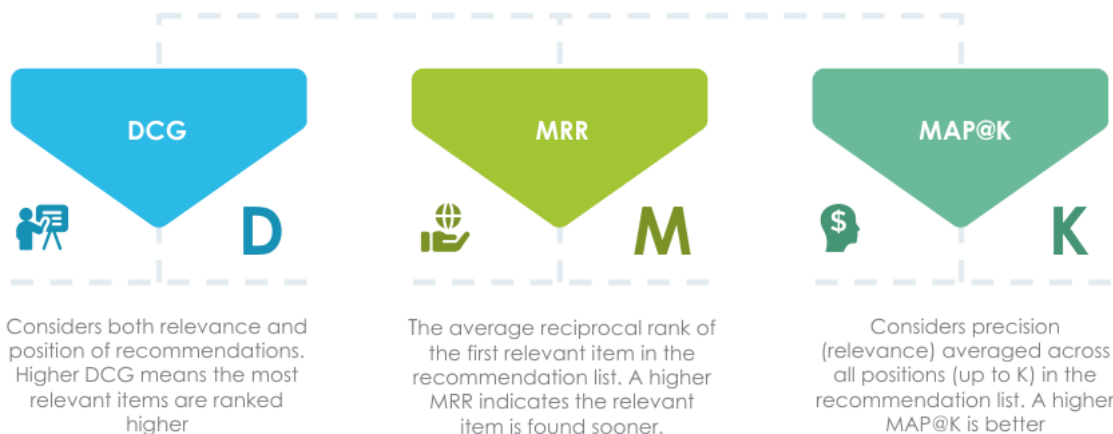
[METRICS FOR SEARCH BOT](#)

METRICS FOR RECOMMENDATION BOT

Metrics for Recommendation

Focus on Quality of Recommendation

Ranking Metrics for Recommendation



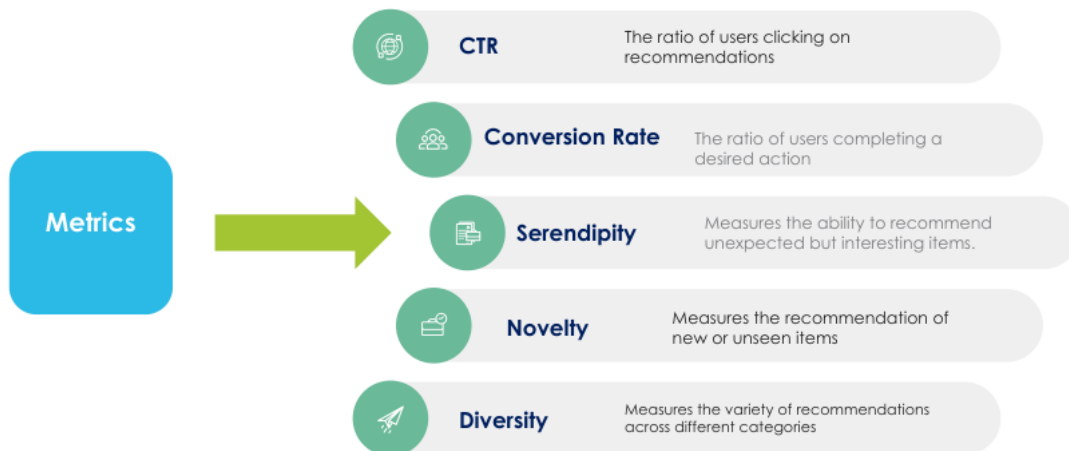
Data
Knobs

[METRICS FOR RECOMMENDATION BOT](#)

BEHAVIORAL METRICS FOR RECOMME

Behavioral Metrics for Recommendation

Focus on user and impact



**Data
Knobs**

[BEHAVIORAL METRICS FOR RECOMME](#)

CRITERIA TO COMPARE LLMS

Criteria to Compare LLM

Present complex data in an easy-to-understand way



01

Size of Model

Bigger size means more parameters



02

Accuracy for Task

Evaluate accuracy for task and determine fitness for scenario



03

Ability to Finetune

How easy it is to fine tune on custom data



04

Training Data Used

What datasets are used? How much is risk in data poison, copyright



05

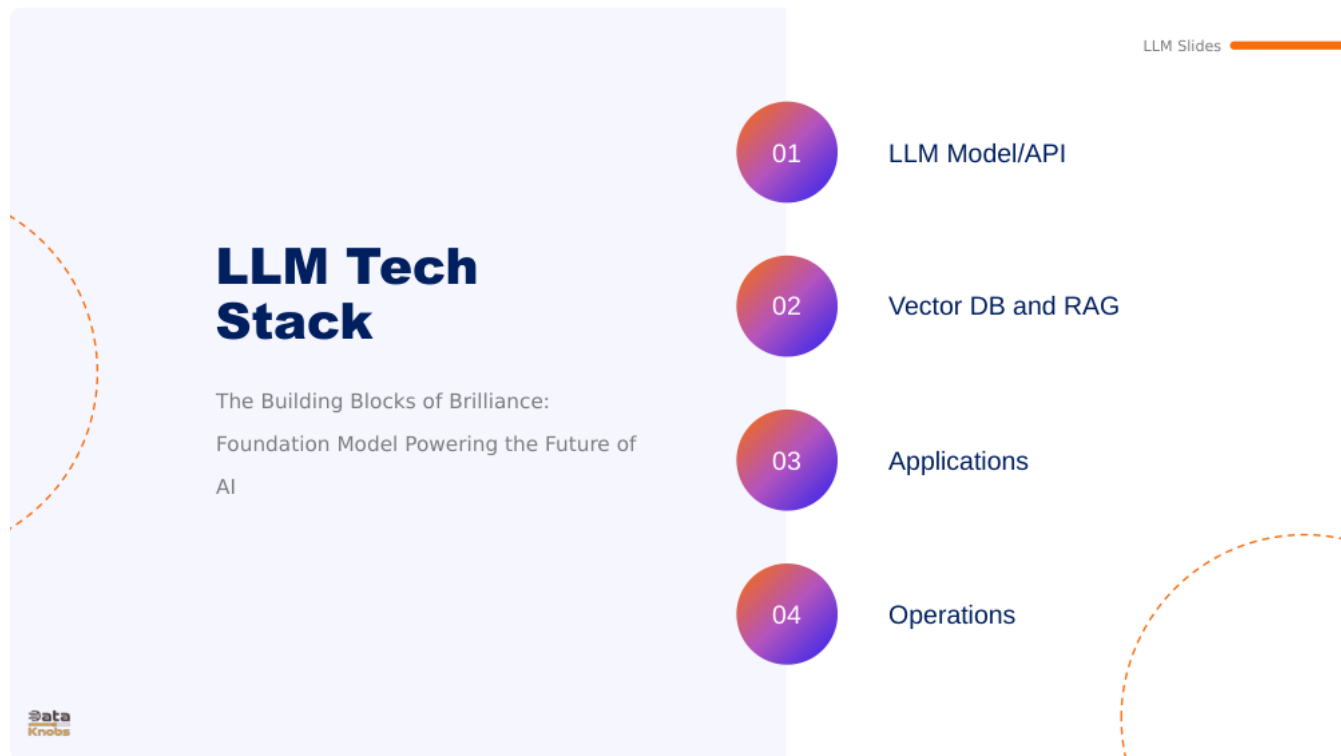
Architecture

What kind of modeling architecture is used to train model?

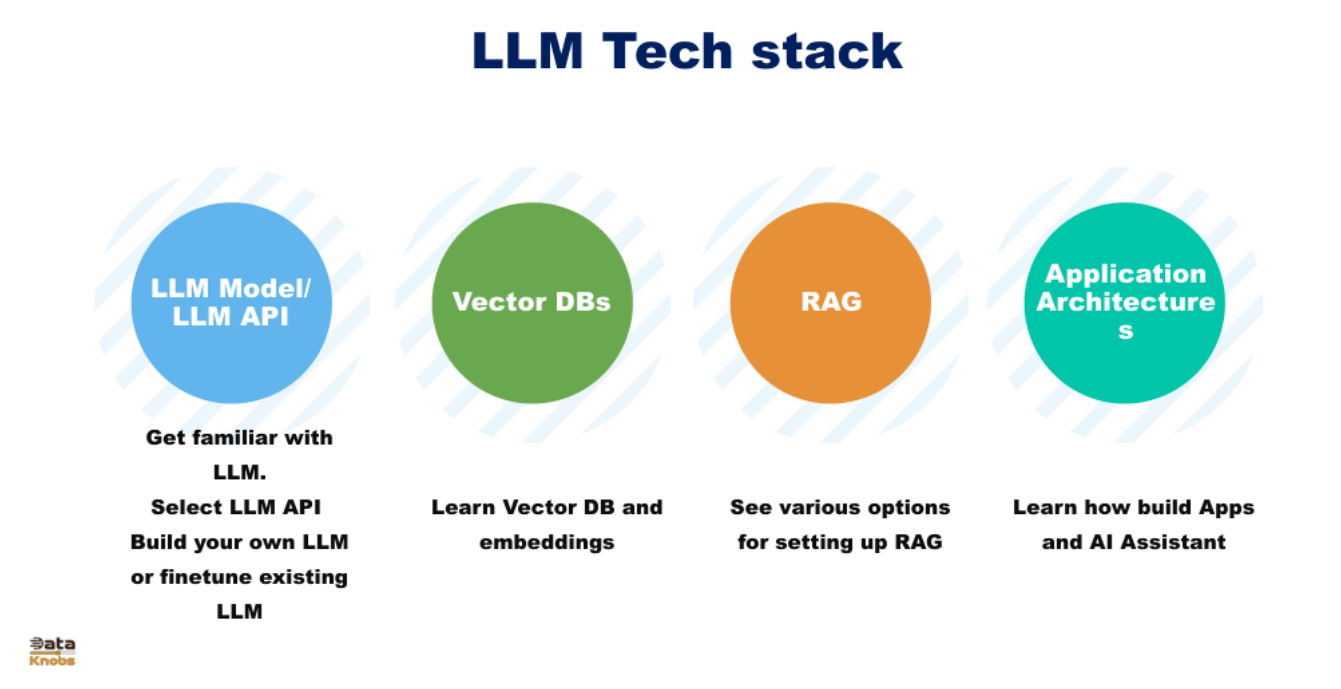
**Data
Knobs**

[CRITERIA TO COMPARE LLMS](#)

LLM TECHNOLOGY SLIDES

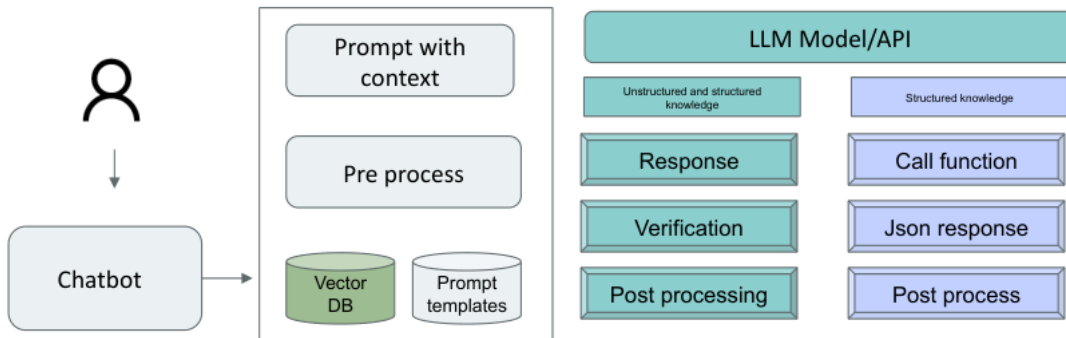
[LLM TECHNOLOGY SLIDES](#)

AI ASSISTANT TECH STACK

[AI ASSISTANT TECH STACK](#)

AI ASSISTANT ARCHITECTURE

LLM based AI Assistant - Architecture



Determine which building blocks you should be building & which building blocks you can get from outside



[AI ASSISTANT ARCHITECTURE](#)

CONSIDERAIONS FOR BOT ARCHITEC

Considerations for Bot building using LLM

- Use proprietary data/latest data/knowledge
- Use different data sources - unstructured, structured, semi structured data
- Ability to Split data in different manner
- Chunk data
- Get context
- Get answer
- Token usage
- Cost
- Context window



[CONSIDERAIONS FOR BOT ARCHITEC](#)

RAG SLIDES

RAG - Retrieval Augmented Generation

RAG is powerful technique for enhancing Large Language Model (LLM) by feeding them with relevant information from external sources.

RAG is an AI framework for retrieving facts from an external knowledge base to ground large language models (LLMs)

- Up-to-date information
- Accurate for tasks
- Give users insight into LLMs' generative process.

When to use : When you need latest info, compliant info and suitable for specific task

[RAG SLIDES](#)**WHEN TO USE RAG**

When to use RAG

- **Domain-specific knowledge:** If your assistant needs to be an expert in a specific domain, RAG can be used to integrate relevant databases or knowledge repositories to enhance its understanding.
- **Accuracy is crucial:** If your LLM assistant needs to provide highly accurate information, especially on factual topics like science, history, or specific procedures, RAG can ensure responses are grounded in real-world knowledge.
- **Combating hallucinations:** LLMs can sometimes make up information, called hallucination. RAG combats this by providing verifiable evidence to support the response.
- **Building trust:** By allowing users to see where the information comes from (think footnotes!), RAG fosters trust and transparency in the assistant's responses.



[WHEN TO USE RAG](#)

WHEN NOT TO USE RAG

When not to use RAG

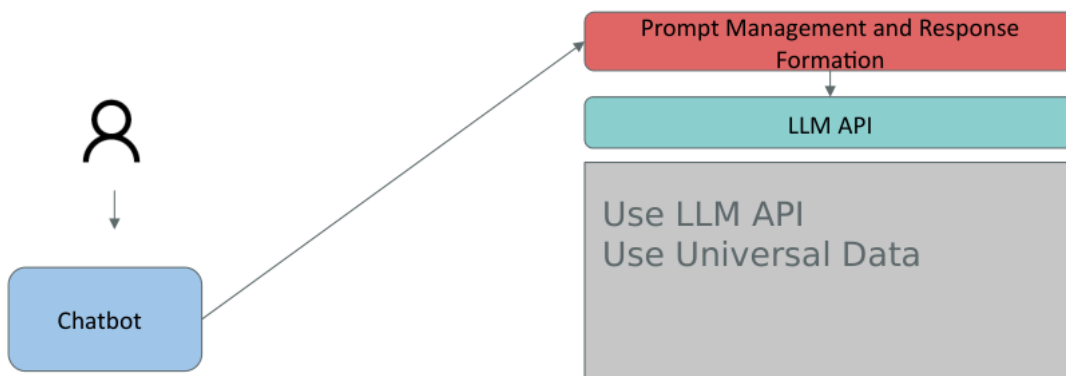
- **Focus is on creative tasks:** If your assistant is primarily used for creative writing, brainstorming, or open ended tasks, RAG might be overkill. The LLM's inherent creativity might be stifled by leaning too heavily on retrieved information.
- **Limited resources:** RAG can add complexity to your LLM system, requiring additional infrastructure for managing the retrieval process. If resources are limited, a simpler LLM approach might be sufficient.
- **Speed is a priority:** RAG introduces an extra step of retrieving information before generating a response. If real-time response is crucial, a standard LLM might be faster.



[WHEN NOT TO USE RAG](#)

AI ASSISTANT WRAPPER

Option 1 - LLM API based Bot

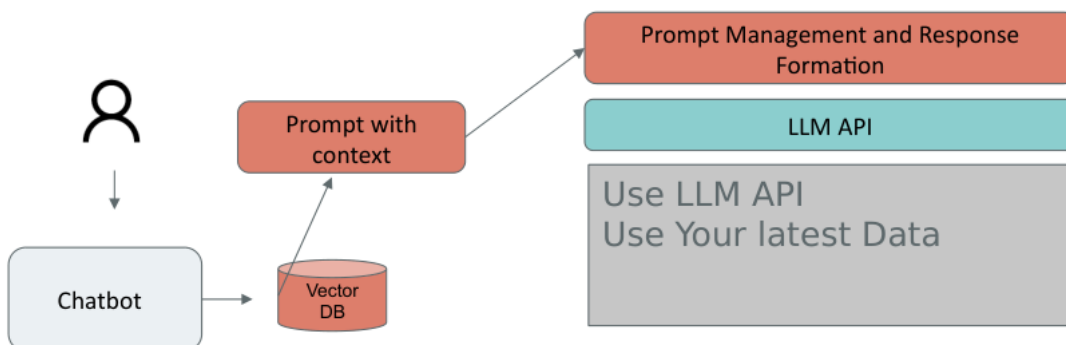


Data
Knobs

[AI ASSISTANT WRAPPER](#)

AI ASSISTANT ON YOUR DATA

Option 2 RAG- LLM based bot with your data

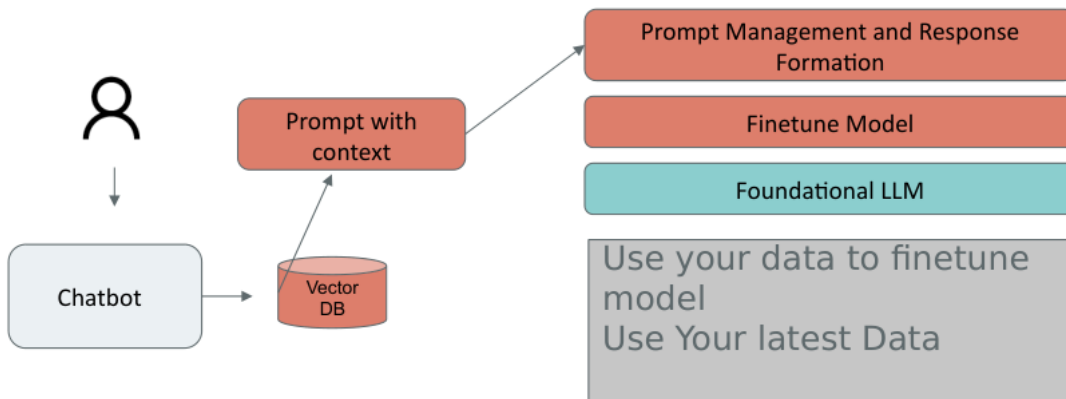


Data
Knobs

[AI ASSISTANT ON YOUR DATA](#)

AI ASSISTANT FINETUNE MODEL

Option 3 Finetune- LLM based bot with your data

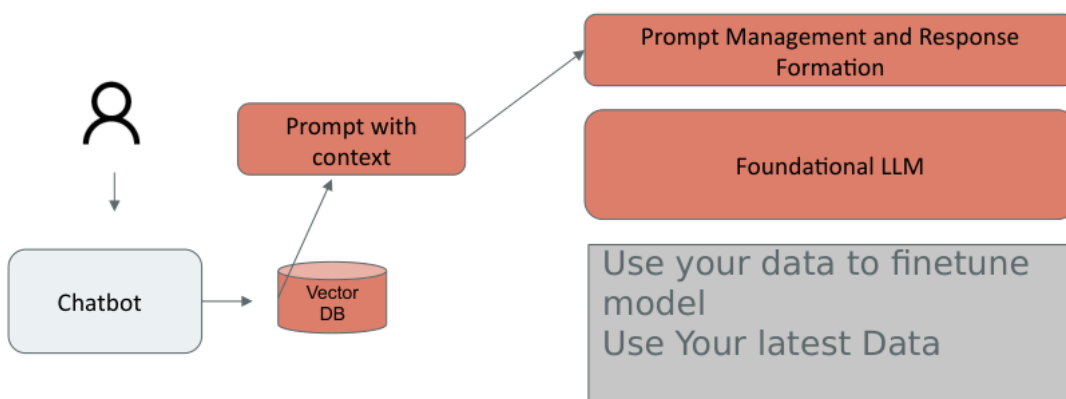


Data
Knobs

[AI ASSISTANT FINETUNE MODEL](#)

AI ASSISTANT CUSTOM MODEL

Option 4 Foundation Model

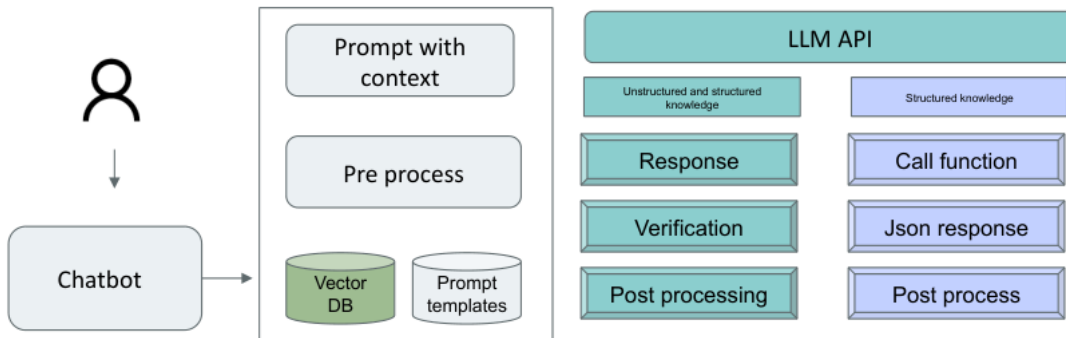


Data
Knobs

[AI ASSISTANT CUSTOM MODEL](#)

AI ASSISTANT BUILDING BLOCKS

LLM based bot - Architecture



Data
Knobs

[AI ASSISTANT BUILDING BLOCKS](#)

LLM FUNCTION CALLING

LLM Model Calling

All large language models share the same interaction model:

- User prompt the model and get back a response.
- Prompt templates help reuse common prompt parts, while allowing parameterized inputs.
- The model maintains no state between interactions, so if we need it to recall things, we must use some form of external memory.
- If we want to connect it with external systems, we need some form of function-calling mechanism and tell the model what functions it has available and how to invoke them.
- Complex tasks require some form of planning – divide and conquer, chain-of-thought reasoning etc.



[LLM FUNCTION CALLING](#)

RAG OVERVIEW SLIDES

RAG - Retrieval Augmented Generation

RAG is an AI framework for retrieving facts from an external knowledge base to ground large language models (LLMs)

- Up-to-date information
- Accurate for tasks
- Give users insight into LLMs' generative process.

When to use : When you need latest info, compliant info and suitable for specific task

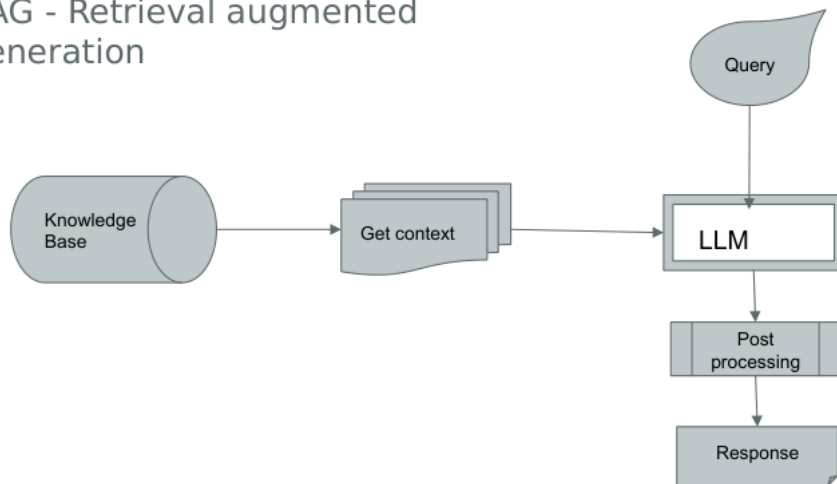


[RAG OVERVIEW SLIDES](#)

RAG ARCHITECTURE SLIDE

RAG Architecture

RAG - Retrieval augmented generation

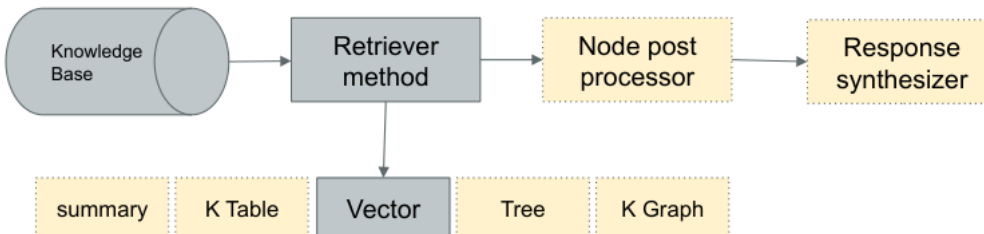


Data
Knobs

[RAG ARCHITECTURE SLIDE](#)

RAG RETRIEVER OPTIONS

RAG Retriever



RAG: Grounding AI in truth.
AI that taps into the your knowledge and use LLM reasoning

Data
Knobs

[RAG RETRIEVER OPTIONS](#)

RAG NODE PROCESSOR

Use of Node Post Processor

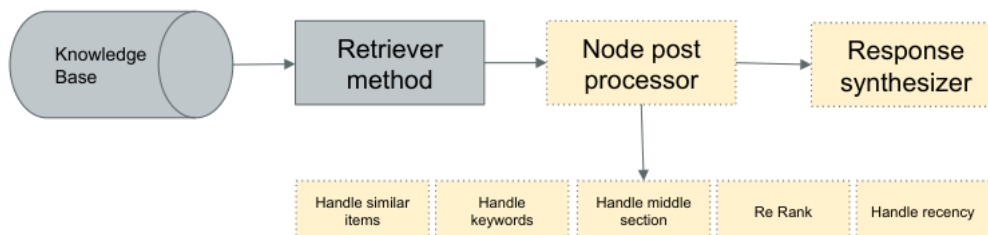
1. Many responses are right when answer are in beginning or in conclusion of pdf. Middle is ignored. Node post processor can be used.
2. Too many similar doc/context are retrieved.
 - a. Scenarios where many similar articles exist in knowledge base
3. You need to exclude certain keyword
 - a. Knowledge base may have offensive/sensitive word and you may want to ignore
 - b. Output that need regulatory compliance can use this to only use right words.

Data
Knobs

[RAG NODE PROCESSOR](#)

RAG NODE POST PROCESSOR

RAG Retriever



Data
Knobs

[RAG NODE POST PROCESSOR](#)

HOW TO FORM RESPONSE IN AI ASS

Use of Response Formation

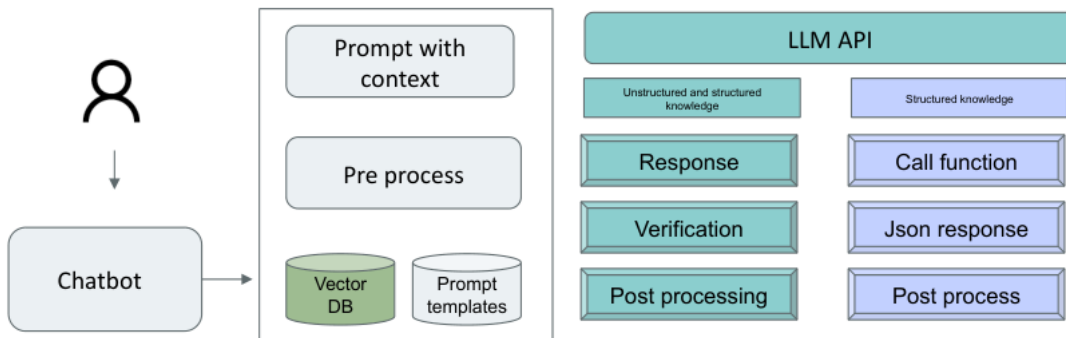
1. Make Compact
2. Convert to Bullet point/Tree structure
3. Refine different part of answers
4. Others



[HOW TO FORM RESPONSE IN AI ASS](#)

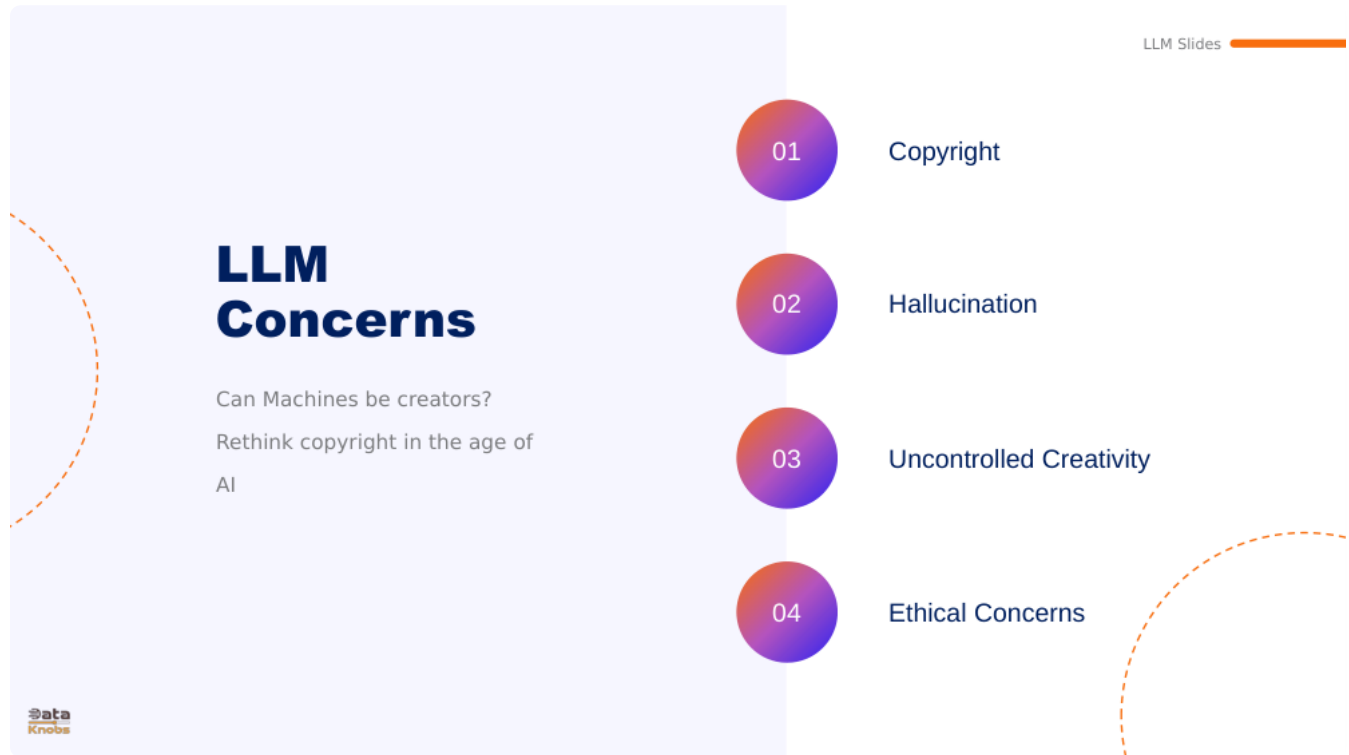
LLM ARCHITECTUE FOR BOT

LLM based bot - Architecture



[LLM ARCHITECTUE FOR BOT](#)

LLM CONCERNS SLIDES



The slide is titled "LLM Concerns" in large, bold, dark blue letters. Below the title, it asks "Can Machines be creators?" and "Rethink copyright in the age of AI". On the right side, there is a vertical list of four concerns, each preceded by a numbered circle (01 to 04) with a purple-to-orange gradient. The concerns are: Copyright, Hallucination, Uncontrolled Creativity, and Ethical Concerns. The slide has a light blue background with a dashed orange arc on the left and right sides. The Data Knobs logo is in the bottom left corner. A small orange bar at the top right indicates the slide is part of a presentation.

LLM Concerns

Can Machines be creators?
Rethink copyright in the age of AI

01 Copyright

02 Hallucination

03 Uncontrolled Creativity

04 Ethical Concerns

Data Knobs

[LLM CONCERNS SLIDES](#)

LLM THREATS SLIDES

LLM Threats

Can we control Large Language Model



[LLM THREATS SLIDES](#)

LLM CHALLENGES SLIDE

Challenges in using LLMs

- Uncontrolled output
- Hallucination
- Resource intensive/large compute
- Prone to Data poison attack
- Copyright issues
- Unethical and unnatural content generation
- Bias
- May product illegal output
- Size of model can make it hard to use, sometimes make it impractical to use

[LLM CHALLENGES SLIDE](#)

LLM ETHICAL CONCERNS SLIDES

Ethical Concerns in using LLMs

- Copyright issue
- Can generate incorrect text/images that seem convincing
- Bias
- Discriminatory text
- Can generate deep fake content
- Can impersonate users
- Can scale cyber attacks



[LLM ETHICAL CONCERNS SLIDES](#)

LLM UNCONTROLLED BEHAVIOR SLID

LLM - Uncontrolled behavior

1. Generative AI model are un supervised/self supervised. There is no labeled truth to evaluate even on training data.
 - a. By design they produce different data even fiction. It is hard to quantify their accuracy.
2. Generative AI model produce output for variety of tasks e.g Q&A, writing articles, summarizing.
 - a. It is hard to provide accuracy/evaluate for variety of tasks
3. Generative AI models are complex deep learning models
 - a. It is hard to explain or test for all conditions



[LLM UNCONTROLLED BEHAVIOR SLID](#)

LLM ETHICAL ISSUES

Ethical Issues

1. Transparency
 - a. Type of data
 - b. Prediction
2. Bias
 - a. Design to handle bias
3. Data privacy
 - a. How to collect, store data
 - b. Know what data you have, what you are collecting
 - c. Know sensitivity of data
 - d. Have clear procedure
 - e. Educate employees
4. IP and copyright violations
 - a. Understand laws, ensure training is compliant
 - b. Ensure generation is compliant
5. Establish incident mgmt
 - a. Reporting and feedback
 - b. Review prompts and output
 - c. Educate consumer about their responsibilities



[LLLM ETHICAL ISSUES](#)

DATA OWNERSHIP ISSUES LLM

Data ownership - open questions

LLM and generative AI image model are train on large universal data from web

Issue Type	Description
Consent	Does company have right to use? Can content owner have a license for reading/non commercial use but not for training Will there be opt-out feature in future
Genuine	Which site has good content and which does not have good content
Poison	What if people start introducing bad data from where LLM learn
Copyright	Who has copyright e.g. A site A publish content. LLM learn from it. A site B use LLM and get content and publish on their site. Now site B has more traffic than site A



[DATA OWNERSHIP ISSUES LLM](#)

LLM GENERATED OUTPUT ISSUES

Data output - open questions

LLM and generative AI image model are train on large universal data from web

Issue Type	Description
Fact	Can produce misleading or factual incorrect results
Harmful	Can produce violent , dangerous content
Fake news	Can generate and propagate fake news
Misrepresentation	Can have Homogeneity and misrepresentation of language/culture



[LLM GENERATED OUTPUT ISSUES](#)

LLM ENVIRONMENT ISSUES

Environmental Issues

LLMs and generative AI require significant compute and can have harmful impact on environment

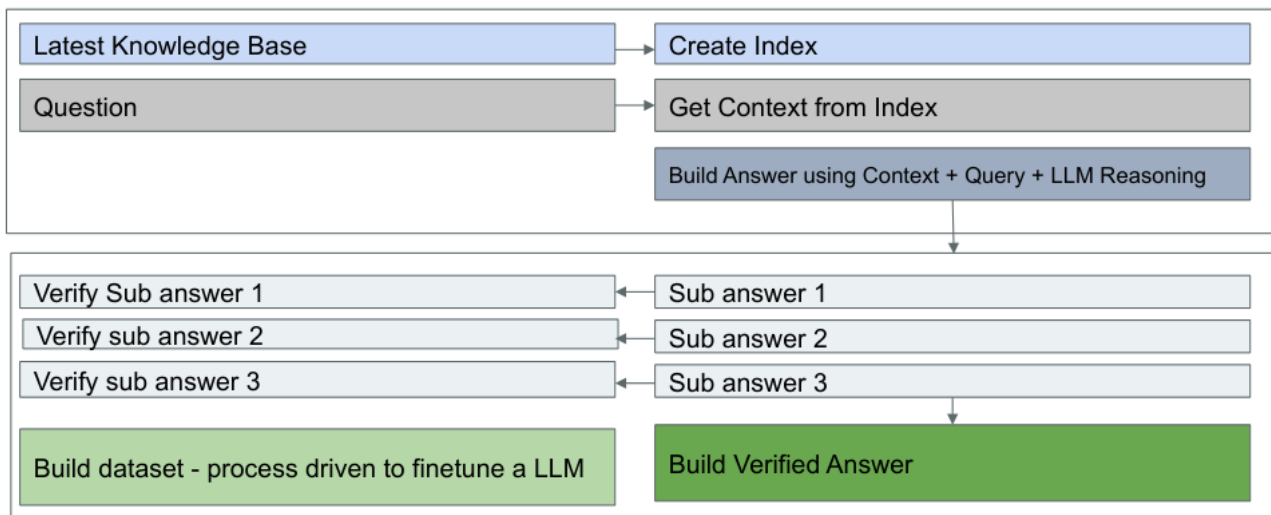
Issue Type	Description
Costly	Costly to generate answer compared to search for routine things
CPU/GPU	Require significant CPU/GPU. Barrier for smaller companies
Rare metals	Rare metals required to crate chips, hardware
Carbon emissions	Generative AI will require more compute and will produce more carbon emission
Natural resources	Running the data center for genrative AI will need significant energy, water for coolant



[LLM ENVIRONMENT ISSUES](#)

ENTERPRISE GRADE ANSWERS AI AS

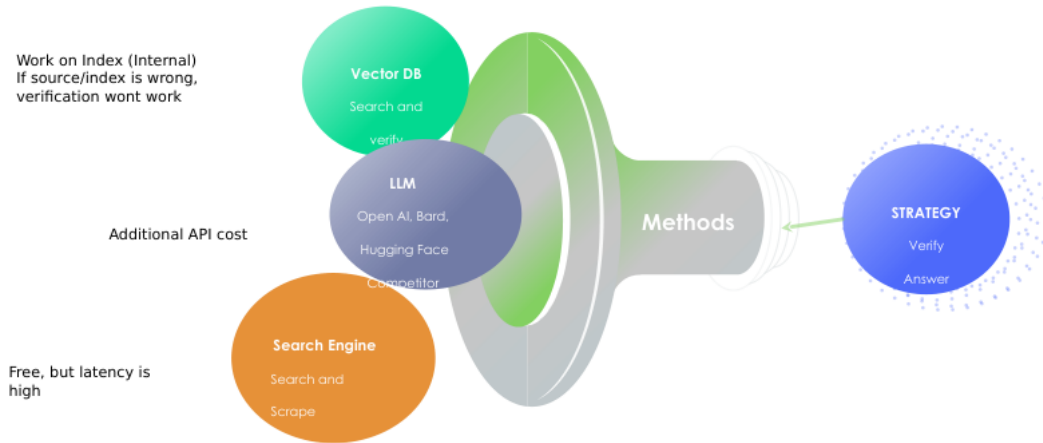
How to provide Enterprise grade answers



[ENTERPRISE GRADE ANSWERS AI AS](#)

APPROACHES TO VERIFY AI ASSIST

3 Approaches to verify Answer



Data
Knobs

APPROACHES TO VERIFY AI ASSIST

EXAMPLE BOT ASSISTANTS

Example Bots

Can Machines be creators?
Rethink copyright in the age of AI

LLMSlides

- 01 Dietitian
- 02 Stocks Earning Call
- 03 Tax Consultant
- 04 Customer Onboarding

EXAMPLE BOT ASSISTANTS

CUSTOMER ONBOARDING BOT

Customer Onboarding Bot

Customer Onboarding Bot need not have to have knowledge for research

Customer Onboarding Bot should be able to answer according to FAQ

- What documents are needed to onboard
- What is fee
- Related topics

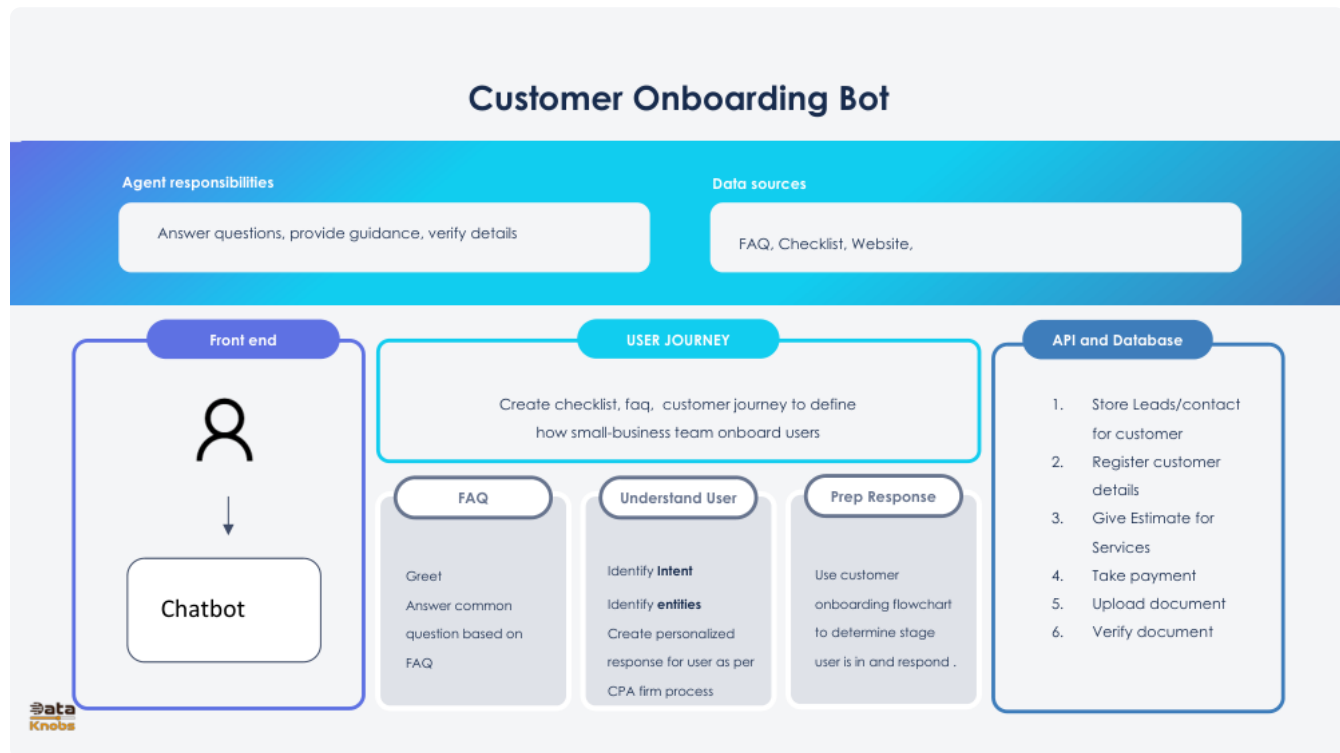
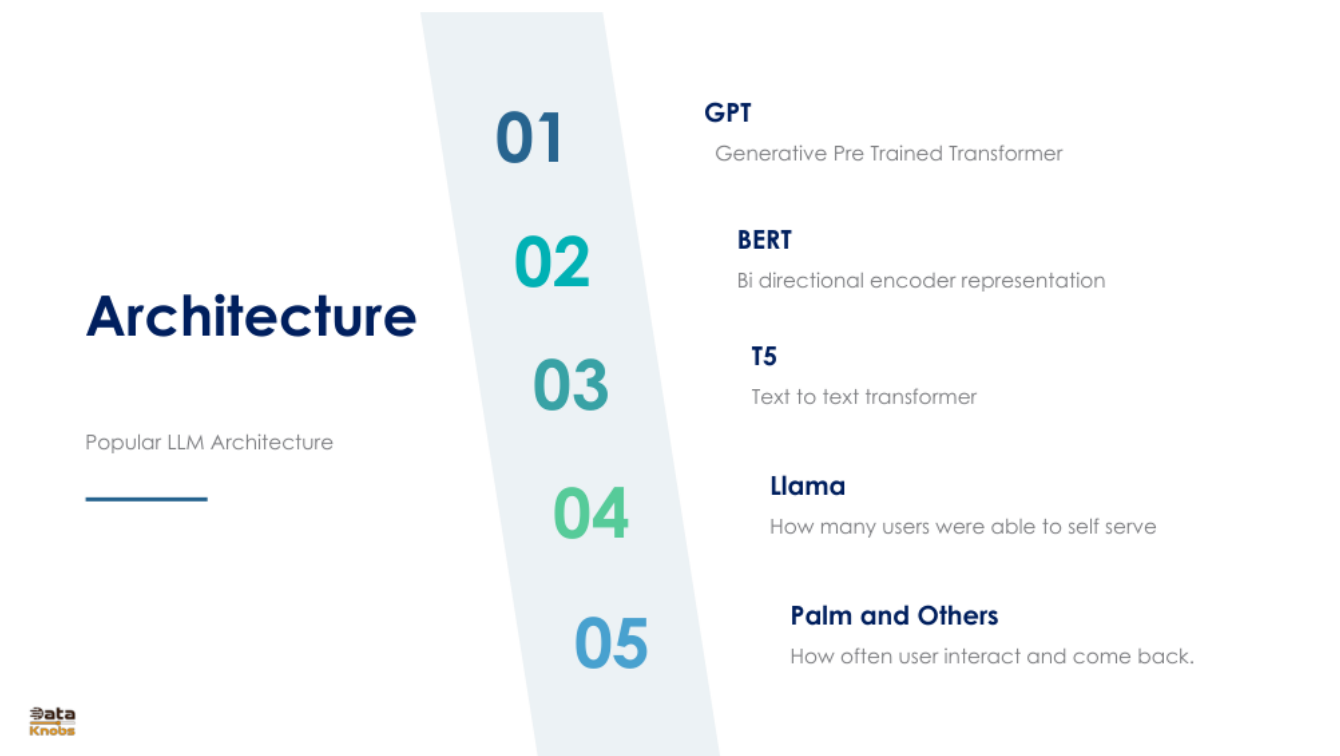
[CUSTOMER ONBOARDING BOT](#)**CUSTOMER ONBOARDING AI ASSISAT**

Customer Onboarding Chatbot



User Guidance	Information Gathering	Personalization	Problem Solving	Advanced
Multi step conversation flow	Verify user	Professional tone	Provide service info	Remember context from past
Conditional branching	Security collect data	Understand user	Provide FAQ, use guide	Learn from user interactions
Walk thru users	Save Data	Provide personalize response	Troubleshoot problems	Improve thru tasks
Provide tutorials links	Integrate with CRM	Give progress update to user	Escalate to live agent	Multilingual support
Share Info	Collect Data to Serve	Serve	Support	Scale and Improve



[CUSTOMER ONBOARDING AI ASSISTANT](#)**ARCHITECTURE SLIDE FOR CUSTOMER**[ARCHITECTURE SLIDE FOR CUSTOMER](#)**SLIDE59**[SLIDE59](#)**LLM TRAINING STEPS**

GPT (Generative Pretrained Transformer)

- GPT is an autoregressive model designed for text generation.
- Key features:
 - - Unidirectional training for sequential data.
 - - Pretrained on large-scale datasets, followed by fine-tuning.
- Variants: GPT, GPT-2, GPT-3, GPT-4.
- Applications: Content creation, summarization, coding assistance.

[LLM TRAINING STEPS](#)

ARCHITECTURE LLM TRAINING

BERT (Bidirectional Encoder Representations from Transformers)

- BERT is designed for bidirectional context understanding.
- Pretrained using Masked Language Modeling (MLM).
- Key applications:
 - - Question answering (e.g., SQuAD).
 - - Text classification.
- Variants: RoBERTa, DistilBERT, ALBERT.

[ARCHITECTURE LLM TRAINING](#)

SLIDE62

T5 (Text-to-Text Transfer Transformer)

- T5 treats every NLP task as a text-to-text problem.
- Unified framework for tasks such as translation, summarization, classification.
- Key innovations:
 - - Pretrained on C4 dataset.
 - - Flexible architecture for diverse NLP tasks.

[SLIDE62](#)

SLIDE63

LLaMA by Meta

LLaMA (Large Language Model Meta AI):

1. Designed for efficient fine-tuning.
2. Targets research and development.
3. Available in multiple sizes (e.g., 7B, 13B, 30B, 65B parameters) to cater to different resource constraints and use cases.
4. Emphasizes lightweight architecture suitable for environments with limited hardware.
5. Built with efficiency in mind, focusing on resource-friendly computation for training and inference.

[SLIDE63](#)

SLIDE64

PaLM By Google

Pathways Language Model

1. Developed by Google as a general-purpose large language model.
2. Scales to billions of parameters (e.g., PaLM 540B), emphasizing cutting-edge performance on large-scale tasks.
3. Excels in few-shot and zero-shot learning capabilities.
4. Incorporates advanced architectural improvements such as efficient attention mechanisms and large-scale distributed training.
5. Ideal for enterprise use cases requiring powerful generative capabilities, including conversational AI, summarization, and code generation.
6. Also used in multimodal tasks such as image and text integration.

[SLIDE64](#)**SLIDE65**

Bloom

Bloom (BigScience Large Open-science Open-access Multilingual Language Model) is an open-source, multilingual large language model.

It was developed as part of the BigScience research initiative, involving collaboration among researchers and organizations worldwide. Designed to support research in natural language processing (NLP) across multiple languages and domains.

a. Transformer-Based Architecture:

- Bloom uses the standard transformer architecture, similar to models like GPT and BERT.
- Composed of stacked layers of attention mechanisms and feedforward neural networks, optimized for text generation and understanding.

b. Multi-Language Support:

- Trained on 46 languages and 13 programming languages.
- Designed to perform well across a diverse set of languages, even those with less available training data.

c. Model Size:

- Bloom comes in various sizes, with the largest model having **176 billion parameters**.
- Comparable in scale to GPT-3 and other large-scale LLMs, enabling it to handle complex NLP tasks.

[SLIDE65](#)**SLIDE66**

Building Blocks

Technical Building Blocks

01

Tokenization

Break Text into smaller units

02

Embedding Layer

Convert tokens into dense vectors

03

Self Attention Mechanism

Calculate importance of each token

04

Positional Encoding

Provide positional information

05

Feed Forward Layers

Apply transformation to enhance model expressiveness.



[SLIDE66](#)

SLIDE67

Tokenization

- Tokenization is the process of breaking text into smaller units (tokens).
- Methods:
 - - Byte Pair Encoding (BPE): Merges frequent pairs of bytes.
 - - WordPiece: Optimizes for the smallest vocabulary with meaningful subwords.
 - - SentencePiece: Allows for model training without pre-tokenized data.



[SLIDE67](#)

SLIDE68

Embedding Layers and Their Significance

- Embedding layers convert tokens into dense vectors of fixed size.
- They capture semantic meanings of words.
- Significance:
 - - Enable efficient computation.
 - - Represent contextual information in numerical form.



[SLIDE68](#)

SLIDE69

Self-attention Mechanism and Multi-head Attention

- Self-attention calculates the importance of each token relative to others.
- Multi-head attention improves model's ability to focus on different aspects of input.
- Advantages:
 - - Handles long-range dependencies.
 - - Enhances contextual understanding.



[SLIDE69](#)

SLIDE70

Positional Encodings

- Positional encodings provide positional information to the model.
- Types:
 - - Sinusoidal: Uses sine and cosine functions.
 - - Learned: Trains positional embeddings along with other parameters.
- Significance: Ensures the model understands sequence order.



[SLIDE70](#)

SLIDE71

Feedforward Layers and Layer Normalization

- Feedforward layers apply non-linear transformations to enhance model's expressiveness.
- Layer normalization stabilizes training by normalizing inputs across features.
- Key Benefits:
 - - Improves convergence.
 - - Reduces internal covariate shift.



[SLIDE71](#)

SLIDE72

LLM Training

Key Concept and Challenges



[SLIDE72](#)**SLIDE73**

Training LLMs

Large Language Models (LLMs) are trained on massive datasets to understand and generate natural language.

Training involves multiple stages:

- Pretraining: Learning general language patterns.
- Fine-tuning: Adapting to specific tasks or domains.

Objectives:

- Improve language understanding and generation capabilities.

[SLIDE73](#)**SLIDE74**

Pretraining Objectives

Masked Language Modeling (MLM):

- Predicts masked tokens in a sentence.
- Example: BERT.

Causal Language Modeling (CLM):

- Predicts the next word based on previous tokens.
- Example: GPT.

Significance:

- MLM focuses on bidirectional context understanding.
- CLM excels in sequential text generation.

[SLIDE74](#)

SLIDE75

Training Datasets and Preprocessing

Training Datasets:

- Web data, books, research articles, code repositories.
- Multilingual and domain-specific datasets.

Preprocessing:

- Tokenization: Converting text into tokens.
- Cleaning: Removing duplicates, noisy data.
- Balancing: Ensuring fair representation of languages and domains.

[SLIDE75](#)**SLIDE76**

Computational Requirements and Optimizations

Computational Requirements:

- High-performance GPUs/TPUs.
- Distributed training across multiple nodes.
- Memory and storage for large-scale datasets.

Optimizations:

- Mixed-precision training to reduce memory usage.
- Gradient checkpointing to save memory.
- Efficient attention mechanisms for scalability.

[SLIDE76](#)

SLIDE77

Challenges in Training LLMs

Bias:

- Bias in training data can propagate to the model.
- Requires strategies like bias detection and mitigation.

Compute Cost:

- Training LLMs is resource-intensive and expensive.
- Emphasis on optimizing resource utilization.

Data Quality:

- Noisy or imbalanced data affects model performance.
- Requires careful dataset curation.

Environmental Impact:

- High energy consumption during training.
- Encourages the adoption of green AI practices.

[SLIDE77](#)

SLIDE78

Fine Tuning LLM

Key Techniques and Evaluation

[SLIDE78](#)**SLIDE79**

Task-Specific Fine-Tuning

- Fine-tuning adapts a pre-trained model to a specific task or domain.
- Examples:
 - Summarization: Refining the model for generating concise text summaries.
 - Sentiment Analysis: Training the model to classify positive or negative sentiment.
- Process:
 - Train on labeled datasets relevant to the task.
- - Update model weights while retaining general language knowledge.

[SLIDE79](#)**SLIDE80**

Few-Shot, Zero-Shot, and Instruction-Tuning

- Few-Shot Learning:
 - Model adapts using a small number of examples.
 - Effective for tasks with limited labeled data.
- Zero-Shot Learning:
 - Model performs tasks without direct examples.
 - Relies on pre-trained knowledge and task descriptions.
- Instruction-Tuning:
 - Fine-tunes models on instruction-response pairs.
 - Enhances model's ability to follow natural language instructions.

[SLIDE80](#)

SLIDE81

Role of Adapters and LoRA

- Adapters:
 - Lightweight modules added to pre-trained models.
 - Enable task-specific fine-tuning without modifying core weights.
 - Reduce computational and storage costs.
- LoRA (Low-Rank Adaptation):
 - Decomposes weight updates into low-rank matrices.
 - Efficiently fine-tunes large models with minimal resource usage.
 - Useful for tasks requiring multiple fine-tuned versions.

[SLIDE81](#)

SLIDE82

Evaluating Fine-Tuned Models

- Metrics:
 - Task-specific: Accuracy, F1-score, BLEU, ROUGE, etc.
 - General: Perplexity for language modeling.
- Considerations:
 - Robustness: Model's performance on unseen data.
 - Bias Detection: Ensuring fairness and neutrality.
 - Efficiency: Balancing performance with computational cost.
- Tools:
 - Use benchmarks like GLUE, SuperGLUE, or task-specific datasets.

[SLIDE82](#)**SLIDE83**

Advanced Topics in LLMs

Exploring Multimodal LLMs: Text, Image, Audio, and Beyond

[SLIDE83](#)

SLIDE84

Introduction to Multimodal LLMs

Definition:

- Multimodal LLMs integrate multiple data modalities (text, images, audio, video) into a unified framework.

Key Advantages:

- Enhanced contextual understanding
- Broader application range across industries
- More natural and human-like interactions

Example Models:

- OpenAI's DALL·E (text-to-image generation)
- CLIP (Contrastive Language–Image Pretraining)

[SLIDE84](#)

SLIDE85

Architecture of Multimodal LLMs

Components:

- Encoders for individual modalities (e.g., CNNs for images, Transformers for text)
- Cross-modal attention layers to integrate modalities
- Decoders to generate outputs (e.g., captions, classifications)

Challenges:

- Aligning heterogeneous data types
- High computational complexity

Innovations:

- Unified architectures combining modalities
- Pretraining on multimodal datasets

[SLIDE85](#)**SLIDE86**

Applications of Multimodal LLMs

Text-to-Image Generation:

- Creating visuals from textual descriptions (e.g., DALL·E)

Video and Audio Analysis:

- Automatic transcription, captioning, and scene analysis

Multimodal Search:

- Query with one modality (e.g., text) to find relevant content in another (e.g., images)

Interactive AI:

- Assistants capable of understanding and responding to multimodal inputs (e.g., speech and visuals)

[SLIDE86](#)

SLIDE87

Multimodal Dataset Examples

Commonly Used Datasets:

- MS COCO: Images with detailed captions
- AudioSet: Audio clips with annotations
- HowTo100M: Videos with instructional captions

Importance of Dataset Quality:

- Ensures accurate alignment between modalities
- Enables robust pretraining and downstream performance

[SLIDE87](#)

SLIDE88

Challenges and Future Directions

Challenges:

- High computational and storage demands
- Alignment errors between modalities
- Ethical concerns: Bias in multimodal datasets

Future Directions:

- Development of more efficient architectures
- Improved cross-modal alignment techniques
- Applications in real-world scenarios (e.g., AR/VR)

[SLIDE88](#)

SLIDE89

Conclusion

- Multimodal LLMs represent the next frontier in AI
- Integration of text, images, audio, and video expands application possibilities
- Ongoing advancements promise to make these models more efficient, accessible, and impactful

[SLIDE89](#)

SLIDE90

Tools and Frameworks for LLMs

Accelerating Model Development
and Deployment

[SLIDE90](#)

SLIDE91

Popular Frameworks for LLMs

PyTorch:

- Dynamic computation graphs for flexibility.
- Extensive library support for NLP and LLMs.

TensorFlow:

- Scalable framework for large-scale training.
- TensorFlow Hub for pre-trained models.

Hugging Face Transformers:

- Simplified access to pre-trained LLMs.
- Tools for fine-tuning and deployment.

[SLIDE91](#)**SLIDE92**

Model Training Pipelines with ML Libraries

ML Libraries for Pipeline Automation:

- scikit-learn: Preprocessing and pipeline building.
- PyTorch Lightning: Simplified training loops.

Key Features:

- End-to-end automation: From data preprocessing to model evaluation.
- Integration with distributed training frameworks.

Example Workflow:

- Data ingestion -> Model training -> Hyperparameter tuning -> Deployment.

[SLIDE92](#)**SLIDE93**

Vector Databases for Contextual Retrieval

Purpose:

- Store and retrieve high-dimensional embeddings efficiently.

Popular Tools:

- Pinecone: Real-time vector search.
- Weaviate: Open-source database with AI integrations.

Applications:

- Retrieval-Augmented Generation (RAG) for context-aware responses.
- Semantic search for improved accuracy.

[SLIDE93](#)**SLIDE94**

Experimentation Platforms and Tools

DataKnobs:

- Platform for A/B testing and experimentation.
- Version control for experiments and datasets.

MLFlow:

- Tracking experiments and managing models.

Benefits:

- Streamlined experimentation and iteration.
- Enhanced reproducibility of experiments.

[SLIDE94](#)

SLIDE95

Conclusion

1. Tools and frameworks streamline LLM development and deployment.
2. Popular frameworks like PyTorch and TensorFlow provide flexibility.
3. Vector databases and experimentation platforms enhance efficiency.
4. Leveraging these tools ensures scalable, effective AI solutions.

[SLIDE95](#)**LARGE LANGUAGE MODELS (LLMs)**

LLMs are a type of artificial intelligence (AI) capable of processing and generating human-like text in response to a wide range of prompts and questions. Trained on datasets of text and code, they can perform various tasks such as:

Generating different creative text formats: poems, code, scripts, musical pieces, emails, letters, etc.

Answering open ended, challenging, or strange questions in an informative way: drawing on their internal knowledge and understanding of the world.

Translating languages: seamlessly converting text from one language to another.

Writing different kinds of creative content: stories, poems, scripts, musical pieces, etc., often indistinguishable from human-written content.

RETRIEVAL AUGMENTED GENERATION (RAG)

RAG is a novel approach that combines the strengths of LLMs with external knowledge sources. It works by:

Retrieval: When given a prompt, RAG searches through an external database of relevant documents to find information related to the query.

Augmentation: The retrieved information is then used to enrich the context provided to the LLM. This can be done by incorporating facts, examples, or arguments in the prompt.

Generation: Finally, the LLM uses the enhanced context to generate a response that is grounded in factual information and tailored to the specific query.

RAG offers several advantages over traditional LLM approaches:

Improved factual accuracy: By anchoring responses in real-world data, RAG reduces the risk of generating false or misleading information.

Greater adaptability: As external knowledge sources are updated, RAG can access the latest information, making it more adaptable to changing circumstances.

Transparency: RAG facilitates a clear understanding of the sources used to generate responses, fostering trust and accountability.

However, RAG also has its challenges:

Data quality: The accuracy and relevance of RAG's outputs depend heavily on the quality of the external knowledge sources.

Retrieval efficiency: Finding the most relevant information from a large database can be computationally expensive.

Integration complexity: Combining two different systems (retrieval and generation) introduces additional complexity in terms of design and implementation.

PROMPT ENGINEERING

Prompt engineering is a crucial technique for guiding LLMs towards generating desired outputs. It involves crafting prompts that:

Clearly define the task: Specify what the LLM should do with the provided information.

Provide context: Give the LLM enough background knowledge to understand the prompt and generate an appropriate response.

Use appropriate language: Frame the prompt in a way that aligns with the LLM's capabilities and training data.

ADVANTAGE OF USING RAG

Better Accuracy: If factual correctness is crucial, RAG can be fantastic. It retrieves information from external sources, allowing the AI to double-check its responses and provide well-sourced answers.

Domain Knowledge: Imagine an AI assistant for medical diagnosis or legal or up to date tax laws. RAG can access medical databases to inform its responses and ensure they align with established medical knowledge.

Reduce Hallucination: LLMs can sometimes fabricate information, a phenomenon called hallucination in which they make up things. RAG mitigates this risk by grounding the response in retrieved data.

Building Trust: By citing sources, RAG fosters trust with users. Users can verify the information and see the reasoning behind the response.

DISADVANTAGES OF USING RAG

Speed is Crucial: RAG involves retrieving information, which can add a slight delay to the response. If real-time response is essential, a trained LLM might be sufficient.

Limited Context: RAG works best when the user's query and context are clear. If the conversation is ambiguous, retrieved information might not be relevant.

Privacy Concerns: If the AI assistant deals with sensitive user data, RAG might raise privacy concerns. External retrievals could potentially expose user information.

WHEN TO FINETUNE LLM

Consider fine-tuning a large language model (LLM) when you want it to perform better at a specific task or adapt to a particular domain. Here are example scenarios where fine-tuning is optimal:

Domain-Specific Nuances: If you need an LLM for financial analysis, legal document review or on medical document analysis - fine-tuning is better. Even though capital has meaning in finance domain. While a pre-trained LLM might understand language, it won't grasp legal, finance or medical specific terms or jargons. Fine-tuning on finance Q&A, document or legal documents will improve the LLM to that specific domain.

Instruction Fine-Tuning: This is a recent advancement where you provide the LLM with instructions or demonstrations alongside training data. This can be useful for tasks where you want the LLM to follow a certain style or format, like writing safety instructions in a specific tone.

Specialized Tasks: Imagine you want an LLM to write different kinds of creative content, like poems or code. Fine-tuning on a dataset of poems can improve its poetry generation skills, while fine-tuning on code samples can enhance its code writing abilities.

However, fine-tuning isn't always the right answer. Here are examples when you should not do fine tuning. In fact you should first use a pre-trained LLM, prompt engineering. Then try RAG. Later consider fine tuning.

General Use Cases: If you need a broad LLM for various tasks, a pre-trained model will do better job. Pre-trained models are versatile, trained on diverse data and can handle many tasks well enough without specific fine-tuning.

Limited Data: Fine-tuning works well when you have a lot of data related to your specific task or domain. If you only add a few records, as the demo shows - fine-tuning might not be effective and could even harm the model's performance.

Knowledge Integration: If your goal is to add proprietary/latest/specific knowledge to the LLM, retrieval-augmented-generation (RAG) is a good approach. In RAG, the LLM retrieves relevant information from a knowledge base. You can use out-of-box RAG. You can further optimize it by summarizing knowledge, creating embeddings with meta-data etc.

FINE TUNING STEPS

How can LLMs be fine-tuned for summarization?

LLMs (Large Language Models) like GPT-3 can be fine-tuned for summarization using the following approaches:

Supervised training - The simplest approach is to fine-tune the LLM using a large dataset of text-summary pairs. The model is trained to generate the corresponding summary given the input text.

This requires a sizable supervised dataset, which can be expensive to create. Public datasets like CNN/DailyMail can be used.

Self-supervised training - The LLM is trained using the original text as input and the first few sentences as the "summary". This creates self-supervision from the data itself.

The model is then fine-tuned on a smaller set of human-written summaries to improve accuracy. This approach requires less labeled data.

Reinforcement learning - The LLM is first trained using autoencoding - to reproduce the input text. Then, rewards are given based on the quality and conciseness of the generated summary.

The model learns to generate better summaries through trial-and-error to maximize these rewards. However, this requires defining a reward function.

Filtering and post-processing - Generated summaries from the LLM can be filtered and refined using techniques like:

- Extracting sentences with the highest similarity to human references
- Removing repetitive sentences
- Combining overlapping content into a single sentence, etc.

This requires minimal fine-tuning of the base LLM but provides less control over the summary style.

Prompting - The LLM can be "prompted" to generate a summary using natural language instructions. For example:

In 2-3 short paragraphs, summarize the main points of the following text:

This relies more on the pre-trained LLM abilities and requires less labeled data. But accuracy tends to be lower.

So in short, there are a variety of approaches to fine-tune LLMs for summarization - from fully supervised to minimally supervised. The choice depends on the available data, required accuracy and custom need.

VERIFY LLM AND AI ASSISTANT ANSWERS

If you are using AI Assistant, you should cross check facts/number given by AI Assistant

Check in Vector DB: If you are using Vector DB/RAG, you can check what value RAG provide. This will help to ensure that response generated by RAG is in line with value stored in vector DB.

Use Second LLM: Other/additional approach is you can ask a smaller question from second or same LLM and see what answer you get. If there is 1 page of text and it says company Dataknobs has revenue of \$78M, you can ask a smaller question "how much revenue Dataknobs has". However you need to consider additional cost of 2nd call? You may have more than one fact and multiple calls may be needed for each fact.

Call to Search Engine: You can run a query on search engine programmatically and check response. However depending on domain this may or may not work. It may require parsing result from search engine.

HOW TO EVALUATE LLM

Method	Description
Perplexity	Perplexity measures how well a language model predicts a sample of text. Lower perplexity indicates better performance.
BLEU Score	BLEU (Bilingual Evaluation Understudy) Score is commonly used to evaluate the quality of machine-translated text by comparing it to human-generated translations.
ROUGE Score	ROUGE (Recall-Oriented Understudy for Gisting Evaluation) Score is used to evaluate the quality of summaries produced by a language model by comparing them to reference summaries.
Human Evaluation	Human evaluation involves having human judges assess the quality of text generated by the language model based on criteria such as fluency, coherence, and relevance.
Word Error Rate (WER)	WER measures the difference between the words generated by the language model and the reference text. Lower WER indicates better performance.

[100K-tokens](#) [Agenda](#) [Ai-assistant-architecture](#) [Ai-assistant-building-blocks](#) [Ai-assistant-custom-model](#) [Ai-assistant-evaluation-metric](#) [Ai-assistant-finetune-on-your-data](#) [Ai-assistant-tech-stack](#) [Ai-assistant-wrapper](#)

DATAKNOBS BLOG

10 USE CASES BUILT

10 USE CASES BUILT BY DATAKNOBS

OUR PRODUCT!

KREATEBOTS

SETUP CHATBOTS IN MINUTES

DataKnobs has developed a wide range of products and solutions powered by Generative AI (GenAI), Agent AI, and traditional AI to address diverse industry needs. These solutions span finance, healthcare, real estate, e-commerce, and more. Click on to see in-depth look at these use cases - Stocks Earning Call Analysis, Ecommerce Analysis with GenAI, Financial Planner AI Assistant, Kreatebots, Kreate Websites, Kreate CMS, Travel Agent Website, Real Estate Agent etc.

AI AGENT TUTORIAL

AGENT AI TUTORIAL

Here are slides and AI Agent Tutorial. Agentic AI refers to AI systems that can autonomously perceive, reason, and take actions to achieve specific goals without constant human intervention. These AI agents use techniques like reinforcement learning, planning, and memory to adapt and make decisions in dynamic environments. They are commonly used in automation, robotics, virtual assistants, and decision-making systems.

BUILD DATAPRODUCTS

HOW DATAKNOBS HELP IN BUILDING DATA PRODUCTS

Building data products using Generative AI (GenAI) and Agentic AI enhances automation, intelligence, and adaptability in data-driven applications. GenAI can generate structured and unstructured data, automate content creation, enrich datasets, and synthesize insights from large volumes of information. This helps in scenarios such as automated report generation, anomaly detection, and predictive modeling.

AI AGENT FOR BUSINESS ANALYSIS

ANALYZE REPORTS, DASHBOARD AND DETERMINE TO-DO

DataKnobs has built an AI Agent for structured data analysis that extracts meaningful insights from diverse datasets such as e-commerce metrics, sales/revenue reports, and sports scorecards. The agent ingests structured data from sources like CSV files, SQL databases, and APIs, automatically detecting schemas and relationships while standardizing formats. Using statistical analysis, anomaly detection, and AI-driven forecasting, it identifies trends, correlations, and outliers, providing insights such as sales fluctuations, revenue leaks, and performance metrics.

KREATEHUB

CREATE NEW KNOWLEDGE WITH PROMPT LIBRARY

At its core, KreateHub is designed to enable creation of new data and the generation of insights from existing datasets. It acts as a bridge between raw data and meaningful outcomes, providing the tools necessary for organizations to experiment, analyze, and optimize their data processes.

BUILD BUDGET PLAN FOR GENAI

CIO GUIDE TO CREATE GENAI BUDGET FOR 2025

CIOs and CTOs can apply GenAI in IT Systems. The guide here describe scenarios and solutions for IT system, tech stack, GenAI cost and how to allocate budget. Once CIO and CTO can apply this to IT system, it can be extended for business use cases across company.

- Pre built front end that you can configure
- Pre built Admin App to manage chatbot
- Prompt management UI
- Personalization app
- Built in chat history
- Feedback Loop
- Available on - GCP,Azure,AWS.
- Add RAG with using few lines of Code.
- Add FAQ generation to chatbot

KREATEBOTS

KREATEWEBSITES

LLM AND AI BASED WEBSITE GENERATI

- AI powered websites to domain search
- Premium Hosting - Azure, GCP,AWS
- AI web designer
- Agent to generate website
- SEO powered by LLM
- Content management system for GenAI
- Buy as SaaS Application or managed serv
- Available on Azure Marketplace too.

KREATEWEBSITES

KREATE CMS

CMS GENAI

- CMS for GenAI
- Lineage for GenAI and Human created cc
- Track GenAI and Human Edited content
- Trace pages that use content
- Ability to delete GenAI content

CMS FOR WEBSITES AND BOTS

GENERATE SLIDES

EFFORTLESS SLIDES CREATION

- Give prompt to generate slides
- Convert slides into webpages
- Add SEO to slides webpages

WHAT IS KREATE AND KREATEPRO**KREATE - BRING YOUR IDEAS TO LIFE**

KREATE empowers you to create things - Dataset, Articles, Presentations, Proposals, Web design, Websites and AI Assistants. Kreate is a platform include set of tools that ignite your creativity and revolutionize the way you work. KreatePro is enterprise version.

WHAT IS KONTROLS**KONTROLS - APPLY CREATIVITY WITH RESPONSIBILITY**

KONTROLS enable adding guardrails, lineage, audit trails and governance. KONTROLS recognizes that different use cases for Gen AI and AI have varying levels of control requirements. KONTROLS provide structure to select right controls.

WHAT IS KNOBS**KNOBS - EXPERIMENTATION AND DIAGNOSTICS**

Well defined tunable parameters for LLM API, LLM fine tuning, Vector DB. These parameters enable faster experimentation and diagnosis for every state of GenAI development - chunking, embedding, upsert into vector DB, retrieval, generation and creating responses for AI Assistant.

KREATE ARTICLES**CREATE ARTICLES AND BLOGS**

Create articles for Blogs, Websites, Social Media posts. Write set of articles together such as chapters of book, or complete book by giving list of topics and Kreate will generate all articles.

KREATE SLIDES**CREATE PRESENTATIONS, PROPOSALS AND PAGES**

Design impactful presentation by giving prompt. Convert your text and image content into presentations to win customers. Search in your knowledge base of presentations and create presentations for different industry. Publish these presentation with one click. Generate SEO for public presentations to index and get traffic.

KREATE WEBSITES**AGENT TO PUBLISH YOUR WEBSITE DAILY**

AI powered website generation engine. It empowers user to refresh website daily. Kreate Website AI agent does work of reading content, website builder, SEO, create light weight images, create meta data, publish website, submit to search engine, generate sitemap and test websites.

KREATE AI ASSISTANTS**BUILD AI ASSISTANT IN LOW CODE/NO CODE****GEN AI SLIDES****CONTENT COMPASS****YOUR STORY WRITING IS AUTOMATED**

- Generate articles
- Generate images
- Generate related articles and images
- Get suggestion what to write next

CONTENT COMPASS**FRACTIONAL CTO FOR GENERATIVE AI AND DATA PRODUCTS****HIRE EXPERTISE WITHOUT COMMITMENT**

- Deliver E2E use case
- Generative AI expertise
- Machine Learning expertise
- Data product building expertise
- Cloud - AWS, GCP, Azure

FRACTIONAL CTO

Set up AI Assistant that give personalized respons to your customers in minutes. Add RAG to AI assistant with minimal code- implement vector DB, create chunks to get contextual answer from your knowlebase. Build quality dataset with us for fine tuning and training a cusom LLM.

CREATE AI AGENT

BUILD AI AGENTS - 5 TYPES

AI agent independently chooses the best actions it needs to perform to achieve their goals. AI agents make rational decisions based on their perceptions and data to produce optimal performance and results. Here are features of AI Agent, Types and Design patterns

DEVELOP DATA PRODUCTS WITH KREATE AND AB EXPERIMENT

DEVELOP DATA PRODUCTS AND CHECK USER RESPONSE THRU EXPERIMENT

As per HBR Data product require validation of both 1. whether algorithm work 2. whether user like it. Builders of data product need to balance between investing in data-building and experimenting. Our product KREATE focus on building dataset and apps , ABExperiment focus on ab testing. Both are designed to meet data product development lifecycle

INNOVATE WITH EXPERIMENTS

EXPERIMENT FASTER AND CHEAPER WITH KNOBS

In complex problems you have to run hundreds of experiments. Plurality of method require in machine learning is extremely high. With Dataknobs approach, you can experiment thru knobs.

RAG FOR UNSTRUCTURED AND STRUCTURED DATA

RAG USE CASES AND IMPLEMENTATION

Here are several value propositions for Retrieval-Augmented Generation (RAG) across different contexts: Unstructured Data, Structred Data, Guardrails.

WHY KNOBS MATTER

KNOBS ARE LEVERS USING WHICH YOU MANAGE OUTPUT

See Drivetrain approach for building data product, AI product. It has 4 steps and levers are key to success. Knobs are abstract mechanism on input that you can control.

MAIN PAGES

RELATED PAGES

EXPLORE MORE

CONTACT US